26.4.1 Developing a Risk Table 538
26.4.2 Assessing Risk Impact 540
26.5 Risk Refinement 542
26.6 Risk Mitigation, Monitoring, and Management 543
26.7 The RMMM Plan 546
26.8 Summary 547

CHAPTER 27 A STRATEGY FOR SOFTWARE SUPPORT 549

27.1 Software Support 550
27.2 Software Maintenance 552
27.2.1 Maintenance Types 553
27.2.2 Maintenance Tasks 554
27.2.3 Reverse Engineering 555
27.3 Proactive Software Support 557
27.3.1 Use of Software Analytics 558
27.3.2 Role of Social Media 559
27.3.3 Cost of Support 559
27.4 Refactoring 560
27.4.1 Data Refactoring 561
24.4.2 Code Refactoring 561
27.4.3 Architecture Refactoring 561
27.5 Software Evolution 562
27.5.1 Inventory Analysis 563
27.5.2 Document Restructuring 564
27.5.3 Reverse Engineering 564
27.5.4 Code Refactoring 564
27.7.5 Data Refactoring 564
27.5.6 Forward Engineering 565
27.6 Summary 565

PART FIVE ADVANCED TOPICS 567

CHAPTER 28 SOFTWARE PROCESS IMPROVEMENT 568

28.1 What Is SPI? 569
28.1.1 Approaches to SPI 569
28.1.2 Maturity Models 570
28.1.3 Is SPI for Everyone? 571

28.2 The SPI Process 571
28.2.1 Assessment and GAP Analysis 572
28.2.2 Education and Training 573
28.2.3 Selection and Justification 573
28.2.4 Installation/Migration 574
28.2.5 Evaluation 575
28.2.6 Risk Management for SPI 575
28.3 The CMMI 576
28.4 Other SPI Frameworks 579
28.4.1 SPICE 579
28.4.2 TickIT Plus 579
28.5 SPI Return on Investment 580
28.6 SPI Trends 580
28.7 Summary 581

CHAPTER 29 EMERGING TRENDS IN SOFTWARE ENGINEERING 583

29.1 Technology Evolution 584
29.2 Software Engineering as a Discipline 585
29.3 Observing Software Engineering Trends 586
29.4 Identifying “Soft Trends” 587
29.4.1 Managing Complexity 588
29.4.2 Open-World Software 589
29.4.3 Emergent Requirements 590
29.4.4 The Talent Mix 591
29.4.5 Software Building Blocks 591
29.4.6 Changing Perceptions of “Value” 592
29.4.7 Open Source 592
29.5 Technology Directions 593
29.5.1 Process Trends 593
29.5.2 The Grand Challenge 594
29.5.3 Collaborative Development 595
29.5.4 Requirements Engineering 596
29.5.5 Model-Driven Software Development 596
29.5.6 Search-Based Software Engineering 597
29.5.7 Test-Driven Development 598
29.6 Tools-Related Trends 599
29.7 Summary 600

CHAPTER 30 CONCLUDING COMMENTS 602

30.1 The Importance of Software—Revisited 603
30.2 People and the Way They Build Systems 603
30.3 Knowledge Discovery 605
30.4 The Long View 606
30.5 The Software Engineer's Responsibility 607
30.6 A Final Comment from RSP 609

APPENDIX 1 An Introduction to UML 611
APPENDIX 2 Data Science for Software Engineers 629
REFERENCES 639
INDEX 659

PREFACE

When computer software succeeds—when it meets the needs of the people who use it, when it performs flawlessly over a long period of time, when it is easy to modify and even easier to use—it can and does change things for the better. But when software fails—when its users are dissatisfied, when it is error prone, when it is difficult to change and even harder to use—bad things can and do happen. We all want to build software that makes things better, avoiding the bad things that lurk in the shadow of failed efforts. To succeed, we need discipline when software is designed and built. We need an engineering approach.

It has been nearly four decades since the first edition of this book was written. During that time, software engineering has evolved from an obscure idea practiced by a relatively small number of zealots to a legitimate engineering discipline. Today, it is recognized as a subject worthy of serious research, conscientious study, and tumultuous debate. Throughout the industry, software engineer has replaced programmer or coder as the job title of preference. Software process models, software engineering methods, and software tools have been adopted successfully across a broad spectrum of industry segments.

Although managers and practitioners alike recognize the need for a more disciplined approach to software, they continue to debate the manner in which discipline is to be applied. Many individuals and companies still develop software haphazardly, even as they build systems to service today's most advanced technologies. Many professionals and students are unaware of modern methods. And as a result, the quality of the software that we produce suffers, and bad things happen. In addition, debate, and controversy about the true nature of the software engineering approach continue. The status of software engineering is a study in contrasts. Attitudes have changed, progress has been made, but much remains to be done before the discipline reaches full maturity.

NEW TO THE NINTH EDITION

The ninth edition of *Software Engineering: A Practitioner's Approach* is intended to serve as a guide to a maturing engineering discipline. The ninth edition, like the eight editions that preceded it, is intended for both students and practitioners, retaining its appeal as a guide for the industry professional and a comprehensive introduction to the student at the upper-level undergraduate or first-year graduate level.

The ninth edition is considerably more than a simple update. The book has been revised and restructured to improve pedagogical flow and emphasize new and important software engineering processes and practices. In addition, we have further enhanced the popular "support system" for the book, providing a comprehensive set of student, instructor, and professional resources to complement the content of the book.

Readers of the past few editions of *Software Engineering: A Practitioner's Approach* will note that the ninth edition has actually been reduced in page length. Our goal was concision, making the book stronger from a pedagogical viewpoint and less daunting for the reader who desires to journey through the entire book. An anecdote, attributed to Blaise Pascal, the famous mathematician and physicist, goes like this: In writing a overly long letter to a friend, Pascal ended with this sentence. "I wanted to write you a shorter letter, but I didn't have the time." As we worked on concision for the ninth edition, we came to appreciate Pascal's words.

The 30 chapters of the ninth edition are organized into five parts. This organization better compartmentalizes topics and assists instructors who may not have the time to complete the entire book in one term.

Part 1, *The Software Process,* presents a variety of different views of software process, considering several important process models and frameworks that allow us to address the debate between prescriptive and agile process philosophies. Part 2, *Modeling,* presents analysis and design methods with an emphasis on object-oriented techniques and UML modeling. Pattern-based design and design for mobility computing applications are also considered. The coverage of user experience design has been expanded in this section. Part 3, *Quality and Security,* presents the concepts, procedures, techniques, and methods that enable a software team to assess software quality, review software engineering work products, conduct SQA procedures, and apply an effective testing strategy and tactics. In addition, we present software security practices that can be inserted into incremental software development models. Part 4, *Managing Software Projects,* presents topics that are relevant to those who plan, manage, and control a software development project. Part 5, *Advanced Topics,* considers software process improvement and software engineering trends. Boxed features are included throughout the book to present the trials and tribulations of a (fictional) software team and to provide supplementary materials about methods and tools that are relevant to chapter topics.

The five-part organization of the ninth edition enables an instructor to "cluster" topics based on available time and student need. An entire one-term course can be built around one or more of the five parts. A software engineering survey course would select chapters from all five parts. A software engineering course that emphasizes analysis and design would select topics from Parts 1 and 2. A testing-oriented software engineering course would select topics from Parts 1 and 3, with a brief foray into Part 2. A "management course" would stress Parts 1 and 4. By organizing the ninth edition in this way,

we have attempted to provide an instructor with a number of teaching options. In every case the content of the ninth edition is complemented by the following elements of the *SEPA, 9/e Support System.*

Additional Resources

A wide variety of resources can be accessed through the instructor website, including an extensive online learning center encompassing problem solutions, a variety of Web-based resources with software engineering checklists, an evolving collection of "tiny tools," and comprehensive case studies. *Professional Resources* provide several hundred categorized web references which allow students to explore software engineering in greater detail, along with a reference library with links to several hundred downloadable references providing an in-depth source of advanced software engineering information. Additionally, a complete online *Instructor's Guide* and supplementary teaching materials along with several hundred PowerPoint slides that may be used for lectures are included.

The *Instructor's Guide for Software Engineering: A Practitioner's Approach* presents suggestions for conducting various types of software engineering courses, recommendations for a variety of software projects to be conducted in conjunction with a course, solutions to selected problems, and a number of useful teaching aids.

When coupled with its online support system, the ninth edition of *Software Engineering: A Practitioner's Approach* provides flexibility and depth of content that cannot be achieved by a textbook alone.

Bruce Maxim has taken the lead in developing new content for the ninth edition of *Software Engineering: A Practitioner's Approach,* while Roger Pressman has served as editor-in-chief as well as providing contributions in select circumstances.

Acknowledgments Special thanks go to Nancy Mead from Software Engineering Institute at Carnegie Mellon University who wrote the chapter on software security engineering; Tim Lethbridge of the University of Ottawa who assisted us in the development of UML and OCL examples and developed the case study that accompanies this book; Dale Skrien of Colby College who developed the UML tutorial in Appendix 1; William Grosky of the University of Michigan–Dearborn who developed the overview of data science in Appendix 2 with his student Terry Ruas; and our Australian colleague Margaret Kellow for updating the pedagogical Web materials that accompany this book. In addition, we would like to thank Austin Krauss for providing insight into software development in the video game industry from his perspective as a senior software engineer.

Special Thanks BRM: I am grateful to have had the opportunity to work with Roger on the ninth edition of this book. During the time I have been working on this book, my son, Benjamin, has become a software engineering manager and my daughter, Katherine, used her art background to create the figures that appear in the book chapters. I am quite pleased to see the adults they have become and enjoy my time with their children